



Práctica 8

Organización y Arquitectura de Computadoras

Docente: Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

Tijuana, Baja California a 16 de octubre de 2025



Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Dificultades durante el desarrollo

Durante el desarrollo de la práctica, el tiempo no lo administre bien y solamente complete la parte de la realización de los corrimientos sin hacer la conversión a valor numérico. Además de que, en el primer punto, me confundí al pensar que si mandaba a un registro de 8bit el carácter y después mandarlo a la pila haciendo un push de ese registro era correcto, sin embargo, el profesor me ayudó en ese tema mencionando que haber mandado algo a ese registro usando mov ya era utilizar la memoria de datos.

Desarrollo

- **Implementar una subrutina que captura una cadena de texto y la invierte sin utilizar memoria de datos, o sea usando pila.**

Se crea una subrutina que irá comparando carácter por carácter con el indicador de fin de la cadena (*), si no es dicho carácter, en vez de guardarlo en memoria se realiza un push del registro utilizado para capturar los caracteres, en este caso AL. Además, para poder calcular la longitud de la cadena, colocamos EDI en 0 para que se vaya incrementando conforme se van mandando caracteres a la pila con el PUSH.

```
; capturar cadena
mov edi, 0          ; se coloca EDI en 0 para usarlo como la longitud de la cadena
ciclo_cap:
    call getche      ; se captura un caracter en AL
    cmp al, '*'      ; se compara AL si ese caracter es el final de la cadena
    je fin           ; de ser cierto, se termina el bucle de captura

    push ax          ; si no es el fin de la cadena, ese caracter se manda a la pila
    inc edi          ; se incrementa EDI porque se capturo un caracter
    jmp ciclo_cap    ; se continua el ciclo

fin:
    call salto
```

```
koria@GioLaptop:/mnt/c/Users/hugog/OneDrive/Documentos/6to Semestre/OAC/Practica8$ ./Practica8
Ingresa una cadena:
otnujnoC*
```



Una vez capturada la cadena y mandado cada caracter a la pila, creamos un loop que termine en ECX y que en cada iteracion vaya haciendo un pop para ir sacando los caracteres de la pila y se vayan imprimiendo. Como se nos solicita imprimirla de manera invertida, no es necesario hacer un desplazamiento en pila usando EBP.

```
; sacar de la pila la cadena
mov ecx, edi ; se manda el contenido de EDI a ECX para usarlo como numero de
iteraciones
ciclo_imprimir:
    pop ax ; se extrae de la pila el caracter previamente capturado,
en este caso, el ultimo antes del *
    call putchar ; se imprime ese caracter
    loop ciclo_imprimir ; se continua el ciclo
call salto
```

```
koria@GioLaptop:/mnt/c/Users/hugog/OneDrive/Documentos/6to Semestre/OAC/Practica8$ ./Practica8
Ingresa una cadena:
otnujnoC*
Cadena invertida:
Conjunto
```



- Implementar una subrutina que solicite el número de corrimientos a la derecha que se le harán a una cadena binaria de texto de 4 bits (ejemplo '0110'), realiza los corrimientos, pasando el bit menos significativo al carry e ingresando en el bit más significativo de esa cadena un '0', luego convierte esa cadena binaria resultante de los corrimientos a valor numérico y la muestra con printHex.

Primero se captura la cantidad de corrimientos utilizando getch y restarle '0' para pasarlo a su valor numérico, con la finalidad de utilizarlo en el loop de corrimientos.

Posterior a eso, guardamos en EDI la dirección de los corrimientos y después enviamos ECX el contenido de EDI, que son las veces que se debe recorrer la cadena en el loop.

Dentro del loop, mandamos la dirección del número a EBX y guardamos en AH el bit menos significativo y le restamos '0' para que tenga su valor numérico. Después, se usa SAHF para mandar el valor de AH a CF.

Una vez guardado el bit menos significativo, se realizan los corrimientos y luego se reemplaza el bit mas significativo con un 0.

```
mov edi, corr      ; se manda la direccion de la cantidad de corrimientos a EDI
mov ecx, [edi]     ; se mandan la cantidad de corrimientos a ECX para usarlo en el
loop
corrimientos:
    mov ebx, num    ; se manda la direccion del numero binario a EDI
    mov ah, [edi + 3] ; se guarda en AL el ultimo bit (menos significativo)
    sub ah, '0'     ; se actualiza el formato a su valor decimal
    SAHF            ; se manda al CF el bit menos significativo guardado en AH

    ; EBX = 0 1 1 0 % = 0 1 1 1
    ;      0 1 2 3 4
    mov al, [ebx + 2] ; bit 2 a bit 3
    mov [ebx + 3], al

    mov al, [ebx + 1] ; bit 1 a bit 2
    mov [ebx + 2], al

    mov al, [ebx + 0] ; bit 0 a bit 1
    mov [ebx + 1], al

    mov byte [ebx + 0], '0' ; se reemplaza el bit mas significativo con un 0
    loop corrimientos
```

```
Ingresa el numero de corrimientos hacia la derecha:
2
Numero de 4 bits original:
0110
Cadena binaria recorrida:
0001
```



Código

```
%include "../Libreria/pc_io.inc" ; se incluye la libreria

section .text
    global _start:

_start:
    ; capturar una cadena e invertirla sin utilizar memoria de datos, dos ciclos, uno para
    guardar en pila y otro para sacar de la pila

    mov edx, cad_prompt
    call newputs
    call salto

    ; capturar cadena
    mov edi, 0 ; se coloca EDI en 0 para usarlo como la longitud de la cadena
    ciclo_cap:
        call getche ; se captura un caracter en AL
        cmp al, '*' ; se compara AL si ese caracter es el final de la cadena
        je fin ; de ser cierto, se termina el bucle de captura

        push ax ; si no es el fin de la cadena, ese caracter se manda a la pila
        inc edi ; se incrementa EDI porque se capturo un caracter
        jmp ciclo_cap ; se continua el ciclo

    fin:
    call salto

    ; se imprime el prompt de cadena invertida
    mov edx, inv_prompt
    call newputs
    call salto

    ; sacar de la pila la cadena
    mov ecx, edi ; se manda el contenido de EDI a ECX para usarlo como numero de iteraciones
    ciclo_imprimir:
        pop ax ; se extrae de la pila el caracter previamente capturado, en
este caso, el ultimo antes del *
        call putchar ; se imprime ese caracter
        loop ciclo_imprimir ; se continua el ciclo
    call salto

; =====
    ; se imprime el mensaje de numero de corrimientos
    mov edx, bin_prompt
    call newputs
    call salto

    ; se captura el numero de corrimientos y se imprime
    ; se captura el primer numero y se guarda
    call getche
    mov ebx, corr
```



```
mov [ebx], al
call salto

; se convierte a numero
mov ebx, corr
sub byte[ebx], '0'

; se imprime el prompt del numero original
mov edx, num_prompt
call newputs
call salto

; se imprime el numero
mov edx, num
call newputs
call salto

mov edx, corr      ; se manda la direccion de la cantidad de corrimientos a EDX
mov ecx, [edx]     ; se mandan la cantidad de corrimientos a ECX para usarlo en el loop
corrimientos:
    mov ebx, num    ; se manda la direccion del numero binario a EDX
    mov ah, [edx + 3] ; se guarda en AL el ultimo bit (menos significativo)
    sub ah, '0'      ; se actualiza el formato a su valor decimal
    SAHF             ; se manda al CF el bit menos significativo guardado en AH

    ; EBX = 0 1 1 0 % = 0 1 1 1
    ;      0 1 2 3 4
    mov al, [ebx + 2] ; bit 2 a bit 3
    mov [ebx + 3], al

    mov al, [ebx + 1] ; bit 1 a bit 2
    mov [ebx + 2], al

    mov al, [ebx + 0] ; bit 0 a bit 1
    mov [ebx + 1], al

    mov byte [ebx + 0], '0' ; se reemplaza el bit mas significativo con un 0
loop corrimientos

; se imprime el prompt del numero binario con corrimientos
mov edx, corrimiento_prompt
call newputs
call salto

; se imprime el numero ya con los corrimientos
mov edx, num
call newputs
call salto

; SYS_EXIT
mov eax, 1
mov ebx, 0
```



```
int 80h

; =====
newputs:
    pushad
    mov esi, 0
    prnt:
        mov al, [edx + esi]    ; se manda a al el caracter incial
        cmp al, '%'           ; se compara al con %
        je finputs            ; si es verdadero, se sale del ciclo

        call putchar          ; se imprime al caracter actual
        inc esi                ; se incrementa el indice
        jmp prnt              ; se reincia el ciclo de impresion

    finputs:
        popad
        ret

salto:
    pushad
    mov al, 13
    call putchar

    mov al, 10
    call putchar
    popad
    ret

printHex:
    pushad
    mov edx, eax
    mov ebx, 0fh
    mov cl, 28
.nxt: shr eax, cl
.msk: and eax, ebx
    cmp al, 9
    jbe .menor
    add al, 7
.menor: add al, '0'
    mov byte [esi], al
    inc esi
    mov eax, edx
    cmp cl, 0
    je .print
    sub cl, 4
    cmp cl, 0
    ja .nxt
    je .msk
.print: mov eax, 4
    mov ebx, 1
    sub esi, 8
    mov ecx, esi
```



```
mov edx, 8
int 80h
popad
ret

section .data
    cad_prompt: db "Ingresa una cadena:%"
    inv_prompt: db "Cadena invertida:%"
    bin_prompt: db "Ingresa el numero de corrimientos hacia la derecha:%"
    corrimiento_prompt: db "Cadena binaria recorrida:%"
    conv_prompt: db "valor convertido: %"
    num: db '0110%'
    num_prompt: db "Numero de 4 bits original:%"
    tabla: db 1,2,4,8,16

section .bss
    cad resb 8
    corr resb 1
    conv resb 1
```

Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)